Mini-Course on Machine Learning for Dynamic Economic Models

Yucheng Yang

University of Zurich and SFI; ERID Visitor, Duke University

Duke University

September 2026

Overview of the Mini-Course

- In the past 10 years, there has been major progress in using machine learning methods to solve and estimate dynamic economic models.
- Three lectures in this mini-course:
  - Lecture 1. Deep Learning for Solving Heterogeneous Agents Models
  - Lecture 2. Structural Reinforcement Learning for Macroeconomics
  - Lecture 3. Deep Learning for Continuous Time Models and Structural Estimation
- Lectures are accompanied by tutorial codes and homework.
- Course repository: github.com/yangycpku/Machine_Learning_Macro_Duke
- Get Google Colab access to run the tutorial notebooks: colab.research.google.com/

Lecture 1. Deep Learning for Solving Heterogeneous Agents Models

Yucheng Yang
University of Zurich and SFI; ERID Visitor, Duke University

Duke Mini-Course on Machine Learning for Dynamic Economic Models
September 2026

Major Development in Macroeconomics

"Major focus of macro over the last 20 years has been the development of models that incorporate rich specifications of heterogeneity and frictions that can simultaneously speak to aggregate outcomes while also addressing a rich set of cross-sectional facts."

- Rogerson (2021).

Some Prominent Examples:

- Uneven dynamics across sectors and population after economic and policy shocks.
- Aggregate impact of monetary/fiscal policy through heterogeneous households/firms.
- Distribution of investors matter for asset prices.

Paradigm shift: Representative agent models $\rightarrow$ Heterogeneous agent models.

New challenge: distribution and its feedback (to decision and aggregates).

HA Model with Aggregate Shocks: Krusell and Smith (1998) Model

- Production economy with a continuum of households: household $i \in [0, 1]$ solves

$$\max_{c_{i,t} \geq 0, a_{i,t+1} \geq \underline{a}} \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(c_{i,t})$$

  subject to budget constraint

$$a_{i,t+1} = w_t y_{i,t} + R_t a_{i,t} - c_{i,t}$$

- Idiosyncratic shocks on employment status $y_{i,t} \in \{y_b, y_g\}$.
- Representative firm produces $Y_t = Z_t F(K_t, \bar{L})$.
- Aggregate shock $Z_t \in \{Z_b, Z_g\}$.
- Markov chain that describes the joint evolution of the exogenous shocks:

$$\pi(z', y' \mid z, y) = \Pr(z_{t+1} = z', y_{t+1} = y' \mid z_t = z, y_t = y)$$

Krusell and Smith (1998) Model (Ct'd)

- Production side: representative firm produces $Y_t = Z_t F(K_t, \bar{L})$.
- Perfect competition + optimization $\Rightarrow$ factor prices:

$$R_t = Z_t \partial_K F(K_t, \bar{L}) - \delta, \qquad w_t = Z_t \partial_L F(K_t, \bar{L})$$

- Capital and labor market clearing $\Rightarrow K_t = \int a_{i,t} di$, $\bar{L} = \int y_{i,t} di$.
- Factor prices are uncertain because of aggregate shock.

Recursive Competitive Equilibrium

- Recursive form of household i's problem: DeepHAM

$$V(a_i, y_i, Z, \mathbf{\Gamma}) = \max_{c_i, a_i'} \left\{ u(c_i) + \beta \mathbb{E} V\left(a_i', y_i', Z', \mathbf{\Gamma}' | y_i, Z\right) \right\}$$

  subject to budget and borrowing constraints $c + a' = w(Z, K(\Gamma))y + R(Z, K(\Gamma))a$, $a' \geq 0$, and perceived law of motion $\mathbf{\Gamma}' = \Psi\left(Z, \mathbf{\Gamma}, Z'\right)$.

- $\mathbf{\Gamma}$: distribution over (a, y) of all households.
- HHs must know $\mathbf{\Gamma}$ to predict factor prices $\Rightarrow$ infinite dimensional $\mathbf{\Gamma}$ is state variable.
- Euler equation with saving policy $a' = g(a, y, Z, \mathbf{\Gamma})$:

$$u_c(Ra + wy - g(a, y, Z, \mathbf{\Gamma})) \geq$$
$$\beta \mathbb{E}\left[R\left(Z', \mathbf{\Gamma}'\right) u_c\left(R\left(Z', \mathbf{\Gamma}'\right) g(a, y, Z, \mathbf{\Gamma}) + w\left(Z', \mathbf{\Gamma}'\right) y' - g\left(g(a, y, Z, \mathbf{\Gamma}), y', Z', \mathbf{\Gamma}'\right)\right)\right]$$

- To solve for g, households need to forecast prices next period, which depend on $\mathbf{\Gamma}'$. $\mathbf{\Gamma}'$ depends on $\mathbf{\Gamma}$ through the equilibrium law of motion Ψ.

Existing Methods

- Heterogeneous agent (HA) models with aggregate shocks are solved with local perturbation method or global Krusell-Smith (KS) method.

	(Local) perturbation method	(Global) KS method
Multiple shocks	Yes	No
Multiple endogenous states	Yes	No
Estimation/Calibration	Yes	No
Large shocks	No	Yes
Risky steady state	No	Yes
Nonlinearity e.g. ZLB	No	Yes

Computational Setup: Krusell-Smith Method

- Curse of dimensionality shows up in recursive form of household i's problem: DeepHAM

$$V(a_i, y_i, Z, \mathbf{\Gamma}) = \max_{c_i, a_i'} \{u(c_i) + \beta \mathbb{E} V(a_i', y_i', Z', \mathbf{\Gamma}' | y_i, Z)\}$$

  subject to budget and borrowing constraints. $\mathbf{\Gamma}$: distribution over (a, y) of all households.

- Krusell-Smith method (KS, 1998; Maliar et al., 2010):
  1. Approximate state: $\hat{s}_i = (a_i, y_i, Z, m_1)$. m_1: first moment of individual asset distribution.
  2. Perceived law of motion (PLM) for m_1: log linear form

$$\log(m_{1,t+1}) = A(Z) + B(Z) \log(m_{1t}).$$

Computational Setup: Krusell-Smith Method

- Curse of dimensionality shows up in recursive form of household i's problem: DeepHAM

$$V(a_i, y_i, Z, \mathbf{\Gamma}) = \max_{c_i, a_i'} \{u(c_i) + \beta \mathbb{E} V(a_i', y_i', Z', \mathbf{\Gamma}' | y_i, Z)\}$$

  subject to budget and borrowing constraints. $\mathbf{\Gamma}$: distribution over (a, y) of all households.

- Krusell-Smith method (KS, 1998; Maliar et al., 2010):
  1. Approximate state: $\hat{s}_i = (a_i, y_i, Z, m_1)$. m_1: first moment of individual asset distribution.
  2. Perceived law of motion (PLM) for m_1: log linear form

$$\log(m_{1,t+1}) = A(Z) + B(Z) \log(m_{1t}).$$

- Very costly in complex HA models with multiple assets or multiple shocks.
- Question: Is it solving the original problem?

Computational Setup: Krusell-Smith Method

- Curse of dimensionality shows up in recursive form of household i's problem: DeepHAM

$$V(a_i, y_i, Z, \mathbf{\Gamma}) = \max_{c_i, a_i'} \{u(c_i) + \beta \mathbb{E} V(a_i', y_i', Z', \mathbf{\Gamma}' | y_i, Z)\}$$

  subject to budget and borrowing constraints. $\mathbf{\Gamma}$: distribution over (a, y) of all households.

- Krusell-Smith method (KS, 1998; Maliar et al., 2010):
  1. Approximate state: $\hat{s}_i = (a_i, y_i, Z, m_1)$. m_1: first moment of individual asset distribution.
  2. Perceived law of motion (PLM) for m_1: log linear form

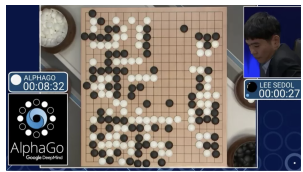
$$\log(m_{1,t+1}) = A(Z) + B(Z) \log(m_{1t}).$$

- Very costly in complex HA models with multiple assets or multiple shocks.
- Question: Is it solving the original problem? No!
- New approach: global solution method using deep learning.

Deep Learning for High Dimensional Models

In recent years, deep learning has achieved remarkable success in many areas, including:

- High dimensional prediction problem (classification or regression).
- High dimensional policy learning.
- Generative models, e.g. LLM.



Deep learning has also led to path-breaking work in high dimensional scientific problems in many disciplines.

Neural Networks: Grid-Free Function Approximator

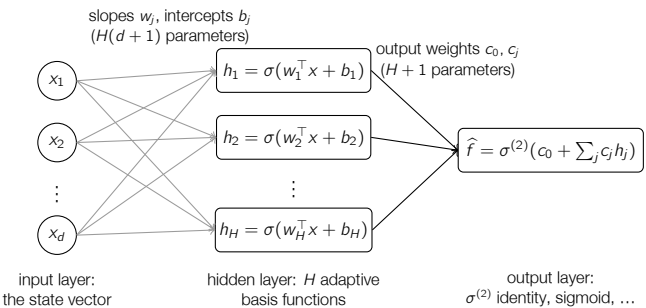
- Neural network (NN): class of grid-free function approximators for high-dim functions
  - grid: n^d points in d dims, e.g. 10^{100} for $n = 10$, $d = 100$: curse of dimensionality
- General form of NN to approximate $\widehat{f}(x; \Theta)$ with $x \in \mathbb{R}^d$. Let $h^{(0)} = x$, and

$$h^{(p)} = \sigma^{(p)}\big(W^{(p)} h^{(p-1)} + b^{(p)}\big), \quad p = 1, \dots, P, \qquad \widehat{f}(x; \Theta) = h^{(P)}$$

  - $W^{(p)}$: $H_p \times H_{p-1}$, $b^{(p)}$: $H_p \times 1$; $p < P$: hidden layers, $p = P$: output layer
  - $\sigma^{(p)}$: element-wise nonlinear activation, e.g. ReLU $\max\{0, u\}$, $\tanh(u)$, sigmoid
  - output $\sigma^{(P)}$ fits the output range: e.g. sigmoid for a rate in $[0, 1]$, $\exp(\cdot)$ if positive
- Unknowns to solve: $\Theta = \{W^{(p)}, b^{(p)}\}_{p=1}^{P}$.
- Widths H_p, depth P: hyperparameters.

One-Hidden-Layer Neural Network

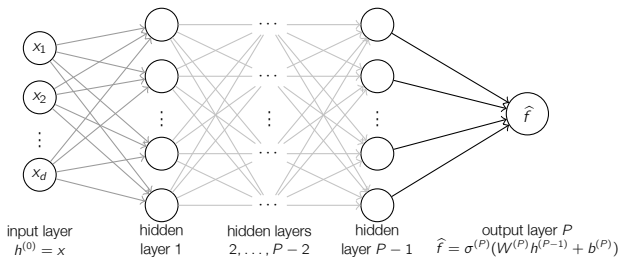
$$\widehat{f}(x;\Theta) = \sigma^{(2)}\Big(c_0 + \sum_{j=1}^{H} c_j\,\sigma\big(w_j^\top x + b_j\big)\Big), \qquad \Theta = \{w_j, b_j, c_j, c_0\} : \; H(d+2)+1 \text{ parameters}$$



Three steps: (1) H affine combos $z_j = w_j^\top x + b_j$; (2) $h_j = \sigma(z_j)$; (3) output $\sigma^{(2)}(c_0 + \sum_j c_j h_j)$. Each $\sigma(w_j^\top x + b_j)$: basis function with learned shape

Deep Neural Network: General P

$$h^{(p)} = \sigma^{(p)}\big(W^{(p)} h^{(p-1)} + b^{(p)}\big), \quad p = 1, \ldots, P, \qquad \widehat{f}(x;\Theta) = h^{(P)}, \quad h^{(0)} = x$$



- Each layer: one-hidden-layer step on previous layer's output: compositional form
- # parameters (width H): $H(d+1) + (P-2)H(H+1) + H + 1$, linear in d;
  $d = 100$, $P = 3$, $H = 64$: 10,689 vs. 10^{100} grid nodes

Optimization: Objective, Sampling, and Stochastic Gradient Descent

- Objective (chosen by economist): expected loss over states $x \sim \mu$, e.g. squared Bellman / Euler residual, or minus lifetime utility

$$\min_{\Theta} \ \mathcal{L}(\Theta) = \mathbb{E}_{x\sim\mu}\big[L(x;\Theta)\big]$$

- Empirical counterpart: sample data $\{x_i\}_{i=1}^{M} \sim \mu$ (e.g. simulated economy, ergodic set):

$$\min_{\Theta} \ \widehat{\mathcal{L}}(\Theta) = \frac{1}{M}\sum_{i=1}^{M} L(x_i;\Theta)$$

- Stochastic gradient descent (SGD) and variants ($\widehat{\mathcal{L}}$ nonconvex, M large):
  - GD: $\Theta \leftarrow \Theta - \eta\,\nabla_\Theta \widehat{\mathcal{L}}(\Theta)$, learning rate η; full pass over M per step
  - minibatch SGD: $\Theta \leftarrow \Theta - \eta\,\frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} \nabla_\Theta L(x_i;\Theta)$, random $\mathcal{B}$: cheap noisy gradient
  - in practice: Adam (momentum + per-coordinate step size)
- Gradients: backprop / autodiff (TensorFlow, PyTorch, JAX) ▸ details ▸ why NNs work

Solving Economic Models with Neural Networks: Three Elements

$$\min_{\Theta} \widehat{\mathcal{L}}(\Theta) = \frac{1}{M} \sum_{i=1}^{M} L(x_i; \Theta)$$

1. Parameterize the functions of interest as neural networks $\widehat{f}(x; \Theta)$: value and policy functions, sometimes price functions. Inputs x: individual states, aggregate shocks, and high-dimensional distribution, even structural parameters!

2. Specify the objective. Maximize a target (e.g., expected lifetime welfare), or minimize a loss: squared residuals of Bellman or Euler equations, market clearing conditions.

3. Choose where to evaluate the objective, the counterpart of designing the grid: a prespecified region, or simulate the economy under the current solution, evaluate on the simulated states, and re-simulate as it improves $\Rightarrow$ accuracy on the ergodic set.

Existing Neural Network Methods Differ Along the Three Elements

Method	Distribution as input	Objective	Where evaluated
Maliar-Maliar-Winant	states of all agents in a simulated panel	Bellman/Euler equation residuals	simulated states
Azinovic-Gaegauf-Scheidegger, DEQN	histogram	All equilibrium conditions (optimality, market clearing)	simulated states
Han-Yang-E, DeepHAM: this lecture	learned generalized moments	maximize simulated lifetime utility	simulated states
Payne-Rebei-Yang, continuous time: Lecture 3	finite-type distribution	residuals of the master equation (PDE)	sampled & simulated states
...	...	...	...

Surveys: Fernández-Villaverde (2026, JEL), Scheidegger (2026).

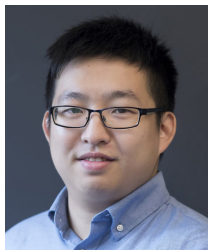
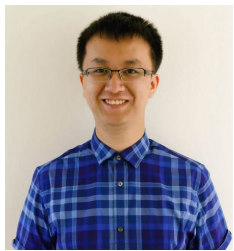
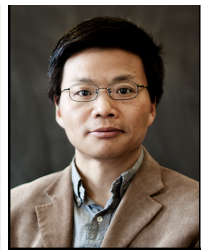
DeepHAM Method

DeepHAM: A global solution method for heterogeneous agent models with aggregate shocks

Jiequn Han
Center for Computational Mathematics, Flatiron Institute

Yucheng Yang
Department of Finance, University of Zurich and SFI

Weinan E
Center for Machine Learning Research and School of Mathematical Sciences, Peking University

DeepHAM: Distribution Representation

- Consider N-agent Krusell-Smith problem (N finite but large). General form of value & policy functions are like (ignore y): Krusell-Smith setup

$$V(a_i; a_1, \ldots, a_{i-1}, a_{i+1}, \ldots, a_N; Z), \quad c(a_i; a_1, \ldots, a_{i-1}, a_{i+1}, \ldots, a_N; Z)$$

DeepHAM: Distribution Representation

- Consider N-agent Krusell-Smith problem (N finite but large). General form of value & policy functions are like (ignore y): Krusell-Smith setup

$$V(a_i; a_1, \ldots, a_{i-1}, a_{i+1}, \ldots, a_N; Z), \quad c(a_i; a_1, \ldots, a_{i-1}, a_{i+1}, \ldots, a_N; Z)$$

- Encode economics in NN architecture: approximate distn with permutation invariant generalized moments $\frac{1}{N}\sum_i \mathcal{Q}(a_i)$, basis function $\mathcal{Q}(\cdot)$ parameterized by (sub) NN:

$$\widetilde{V}(a_i; \frac{1}{N}\sum_i \mathcal{Q}_1(a_i), \ldots, \frac{1}{N}\sum_i \mathcal{Q}_J(a_i); Z)$$

$$\widetilde{c}(a_i; \frac{1}{N}\sum_i \tilde{\mathcal{Q}}_1(a_i), \ldots, \frac{1}{N}\sum_i \tilde{\mathcal{Q}}_J(a_i); Z)$$

DeepHAM: Distribution Representation

- Consider N-agent Krusell-Smith problem (N finite but large). General form of value & policy functions are like (ignore y): Krusell-Smith setup

$$V(a_i; a_1, \ldots, a_{i-1}, a_{i+1}, \ldots, a_N; Z), \quad c(a_i; a_1, \ldots, a_{i-1}, a_{i+1}, \ldots, a_N; Z)$$

- Encode economics in NN architecture: approximate distn with permutation invariant generalized moments $\frac{1}{N}\sum_i \mathcal{Q}(a_i)$, basis function $\mathcal{Q}(\cdot)$ parameterized by (sub) NN:

$$\widetilde{V}(a_i; \frac{1}{N}\sum_i \mathcal{Q}_1(a_i), \ldots, \frac{1}{N}\sum_i \mathcal{Q}_J(a_i); Z)$$

$$\widetilde{c}(a_i; \frac{1}{N}\sum_i \tilde{\mathcal{Q}}_1(a_i), \ldots, \frac{1}{N}\sum_i \tilde{\mathcal{Q}}_J(a_i); Z)$$

- Special case: $\mathcal{Q}(a) = a$ yields the first moment.

DeepHAM: Distribution Representation

- Consider N-agent Krusell-Smith problem (N finite but large). General form of value & policy functions are like (ignore y): Krusell-Smith setup

$$V(a_i; a_1, \ldots, a_{i-1}, a_{i+1}, \ldots, a_N; Z), \quad c(a_i; a_1, \ldots, a_{i-1}, a_{i+1}, \ldots, a_N; Z)$$

- Encode economics in NN architecture: approximate distn with permutation invariant generalized moments $\frac{1}{N}\sum_i \mathcal{Q}(a_i)$, basis function $\mathcal{Q}(\cdot)$ parameterized by (sub) NN:

$$\widetilde{V}(a_i; \frac{1}{N}\sum_i \mathcal{Q}_1(a_i), \ldots, \frac{1}{N}\sum_i \mathcal{Q}_J(a_i); Z)$$

$$\widetilde{c}(a_i; \frac{1}{N}\sum_i \tilde{\mathcal{Q}}_1(a_i), \ldots, \frac{1}{N}\sum_i \tilde{\mathcal{Q}}_J(a_i); Z)$$

- Special case: $\mathcal{Q}(a) = a$ yields the first moment.
- Algorithm solves generalized moments (GMs) that matter most for policy and value functions. ("numerically determined sufficient statistics")

DeepHAM: Distribution Representation

- Consider N-agent Krusell-Smith problem (N finite but large). General form of value & policy functions are like (ignore y): Krusell-Smith setup

$$V(a_i; a_1, \ldots, a_{i-1}, a_{i+1}, \ldots, a_N; Z), \quad c(a_i; a_1, \ldots, a_{i-1}, a_{i+1}, \ldots, a_N; Z)$$

- Encode economics in NN architecture: approximate distn with permutation invariant generalized moments $\frac{1}{N}\sum_i \mathcal{Q}(a_i)$, basis function $\mathcal{Q}(\cdot)$ parameterized by (sub) NN:

$$\widetilde{V}(a_i; \frac{1}{N}\sum_i \mathcal{Q}_1(a_i), \ldots, \frac{1}{N}\sum_i \mathcal{Q}_J(a_i); Z)$$

$$\widetilde{c}(a_i; \frac{1}{N}\sum_i \tilde{\mathcal{Q}}_1(a_i), \ldots, \frac{1}{N}\sum_i \tilde{\mathcal{Q}}_J(a_i); Z)$$

- Special case: $\mathcal{Q}(a) = a$ yields the first moment.
- Algorithm solves generalized moments (GMs) that matter most for policy and value functions. ("numerically determined sufficient statistics")
- GMs provide interpretability on how heterogeneity matters.

DeepHAM Algorithm: General Procedure

- Formulate discrete time N-agent HA models, solve value and policy functions parameterized by neural nets $V(s)$, $c(s)$, $s = (a_i, y_i, Z, \mathbf{\Gamma})$; NN parameters Θ^V, Θ^C.
- Parameterize two parts of mapping:
  1. Distribution $\mathbf{\Gamma} \mapsto J$ generalized moments $\frac{1}{N}\sum_i \mathcal{Q}_j(a_i)$.
  2. $(a_i, y_i, Z, \{\frac{1}{N}\sum_i \mathcal{Q}_j(a_i)\}) \mapsto c, V$.

DeepHAM Algorithm: General Procedure

- Formulate discrete time N-agent HA models, solve value and policy functions parameterized by neural nets $V(s)$, $c(s)$, $s = (a_i, y_i, Z, \mathbf{\Gamma})$; NN parameters Θ^V, Θ^C.
- Parameterize two parts of mapping:
  1. Distribution $\mathbf{\Gamma} \mapsto J$ generalized moments $\frac{1}{N}\sum_i \mathcal{Q}_j(a_i)$.
  2. $(a_i, y_i, Z, \{\frac{1}{N}\sum_i \mathcal{Q}_j(a_i)\}) \mapsto c, V$.
- Iteratively update value and policy functions. In each iteration:
  1. Simulate stationary distribution with the latest policy.
  2. Given policy function, update value function with supervised learning (regression).
  3. Given value function, optimize policy function on simulated paths.

DeepHAM: Value Function Learning

Define cumulative utility for HH i up to t:

$$\widetilde{U}_{i,t} = \sum_{\tau=0}^{t} \beta^{\tau} u\left(c_{i,\tau}\right).$$

In iteration k, given policy function $\mathcal{C}^{(k-1)}(s)$:

1. Sample states s from the stationary distribution. Then the value of each state s can be approximately calculated as cumulative utility in the following T (T large enough) periods following policy $\mathcal{C}^{(k-1)}(s)$:

$$\widetilde{V}^{(k)}(s) \approx \mathbb{E}\widetilde{U}_T = \mathbb{E}\sum_{\tau=0}^{T} \beta^{\tau} u\left(c_{i,\tau}\right)$$

2. Learn value function $V^{(k)}(s)$ parameterized by deep neural networks with regression/supervised learning. Function input = states, output = truncated value. Sample: states in the simulation.

DeepHAM: Policy Function Optimization

In iteration k, given $V^{(k)}(s)$, optimize policy $\mathcal{C}^{(k)}(s)$ on simulated paths.

Fictitious play: in N-agent competitive equilibrium problem, when solving agent i's problem, fix other agents' policy from last "play". Iterate the following:

1. At "play" $\ell+1$, last play's policy $\mathcal{C}^{(k,\ell)}(s)$ is known.
2. For agent $i=1$, update her optimal policy $\mathcal{C}^{(k,\ell+1)}(s)$ according to:

$$\max_{\mathcal{C}^{(k,\ell+1)}(s)} \mathbb{E}_{\mu(\mathcal{C}^{(k-1)}),\mathcal{E}} \left(\sum_{t=0}^{T} \beta^t u\left(c_{i,t}\right) + \beta^T V^{(k)}(s_{i,T}) \right)$$

   subject to others all following $\mathcal{C}^{(k,\ell)}(s)$ in the first T periods.
3. All agents adopt the new policy $\mathcal{C}^{(k,\ell+1)}(s)$ in "play" $\ell+1$.

DeepHAM: Policy Function Optimization

In iteration k, given $V^{(k)}(s)$, optimize policy $\mathcal{C}^{(k)}(s)$ on simulated paths.

Fictitious play: in N-agent competitive equilibrium problem, when solving agent i's problem, fix other agents' policy from last "play". Iterate the following:

1. At "play" $\ell+1$, last play's policy $\mathcal{C}^{(k,\ell)}(s)$ is known.
2. For agent $i=1$, update her optimal policy $\mathcal{C}^{(k,\ell+1)}(s)$ according to:

$$\max_{\mathcal{C}^{(k,\ell+1)}(s)} \mathbb{E}_{\mu(\mathcal{C}^{(k-1)}),\mathcal{E}} \left(\sum_{t=0}^{T} \beta^t u\left(c_{i,t}\right) + \beta^T V^{(k)}(s_{i,T}) \right)$$

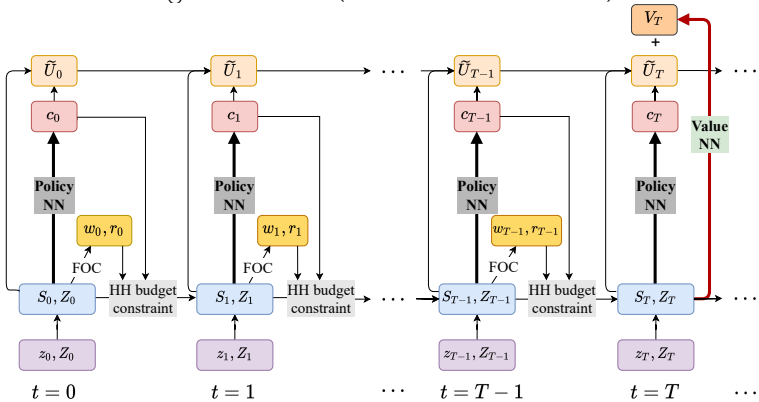
   subject to others all following $\mathcal{C}^{(k,\ell)}(s)$ in the first T periods.

3. All agents adopt the new policy $\mathcal{C}^{(k,\ell+1)}(s)$ in "play" $\ell+1$.

Optimization solved on Monte Carlo simulation with N agents on a large number of sample paths in a computational graph.

Computational Graph for Policy Function Optimization

$$\max_{\Theta^C} \mathbb{E}_{\mu(\mathcal{C}^{(k-1)}),\mathcal{E}} \left(\widetilde{U}_{i,T} + \beta^T V_{\text{NN}}(s_{i,T}; \Theta^V) \right)$$



Budget constraint $a_{i,t+1} = R_t a_{i,t} + w_t y_{i,t} - c_{i,t}$. $s_t = (a_{i,t}, y_{i,t}, Z_t, \mathbf{\Gamma}_t)$. Cumulative utility $\widetilde{U}_{i,t} = \sum_{\tau=0}^{t} \beta^\tau u(c_{i,\tau})$

Remarks on optimization over simulated paths

- Agents formulate expectation over future via long simulation: no perceived law of motion needed.
  - Similar to the idea of reinforcement learning.
  - No reliance on equilibrium conditions: solves models for which FOCs are hard to derive (e.g. non-convex problem, planner's problem).
  - Can be extended to study problems beyond full information rational expectation equilibrium (more in Lecture 2 on Structural Reinforcement Learning)

Remarks on optimization over simulated paths

- Agents formulate expectation over future via long simulation: no perceived law of motion needed.
  - Similar to the idea of reinforcement learning.
  - No reliance on equilibrium conditions: solves models for which FOCs are hard to derive (e.g. non-convex problem, planner's problem).
  - Can be extended to study problems beyond full information rational expectation equilibrium (more in Lecture 2 on Structural Reinforcement Learning)
- Can easily extend to constrained efficiency (planner's) problem (mean field control).
  - Competitive equilibrium: fictitious play.
  - Constrained efficiency: optimize all agents' policy together.

Remarks on optimization over simulated paths

- Agents formulate expectation over future via long simulation: no perceived law of motion needed.
  - Similar to the idea of reinforcement learning.
  - No reliance on equilibrium conditions: solves models for which FOCs are hard to derive (e.g. non-convex problem, planner's problem).
  - Can be extended to study problems beyond full information rational expectation equilibrium (more in Lecture 2 on Structural Reinforcement Learning)
- Can easily extend to constrained efficiency (planner's) problem (mean field control).
  - Competitive equilibrium: fictitious play.
  - Constrained efficiency: optimize all agents' policy together.
- Finite agent approximation + fictitious play: can be used to solve strategic equilibrium.

Implementation Code of DeepHAM

- Our (official) version in TensorFlow: `https://github.com/frankhan91/DeepHAM`
- PyTorch version by Marko Irisarri (University of Manchester): `https://github.com/markoirisarri/UnofficialDeepHAMPytorchImplementation`
- JAX version by Chiyuan Wang (Peking University): `https://github.com/leafDancer/DeepHAMX`

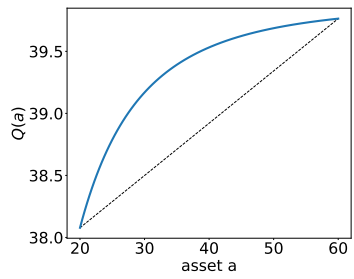
Accuracy Results for Krusell-Smith Problem

Method and Moment Choice	Bellman error	Std of error
KS Method (Maliar et al., 2010)	0.0253	0.0002
DeepHAM with 1st moment	0.0184	0.0023
DeepHAM with 1 generalized moment	0.0151	0.0015

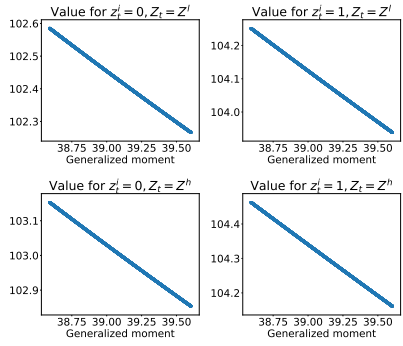
Definition of Bellman Error

- Highly accurate compared to Krusell-Smith (KS) method. solution comparison
- Even only with first moment as model input, DeepHAM outperforms KS method due to better capture of nonlinearity.
- Generalized moment yields more accurate solution than the first moment, as it extracts more relevant information.

Interpretation of the Generalized Moment (GM)



(a) Plot of $\mathcal{Q}_1(a)$



(b) Map $\frac{1}{N}\sum_i \mathcal{Q}_1(a_i)$ to value fn

- Basis function concave in asset, value function is linear wrt the GM.
- Heterogeneity matters! Unanticipated redistributive policy shock: asset from rich to poor HH $\Rightarrow$ generalized moment$\uparrow\Rightarrow$ unshocked agent welfare$\downarrow$.
- KS method implies no effect, as the first moment does not change.

DeepHAM for Constrained Efficiency Problem

- Constrained efficiency problem: planner's allocation in incomplete market.
- Important "second best" allocation, but hard to solve in HA models.
- Literature only solves for HA models without aggregate shocks (Davila, Hong, Krusell, Rios-Rull, 2012; Nuño and Moll, 2018).

DeepHAM for Constrained Efficiency Problem

- Constrained efficiency problem: planner's allocation in incomplete market.
- Important "second best" allocation, but hard to solve in HA models.
- Literature only solves for HA models without aggregate shocks (Davila, Hong, Krusell, Rios-Rull, 2012; Nuño and Moll, 2018).
- DeepHAM solves constrained efficiency problem as easily as the competitive equilibrium: just remove the fictitious play step.
- We solve constrained efficiency problem of Davila et al. (2012), and that with aggregate shocks and countercyclical unemployment risk.
- It takes DeepHAM 20 minutes to solve Davila et al. (2012) on GPU, which takes conventional methods $>$ 10 hours on CPU.

Constrained Efficiency for HA Models w or w/o Agg Shock

	No aggregate shock		Aggregate shock	
	Market	Constrained Opt.	Market	Constrained Opt.
Average assets	30.635	119.741	34.296	95.811
Wealth Gini	0.864	0.862	0.812	0.878
Consumption Gini	0.615	0.386	0.578	0.388

Findings:

1. Both models: with utilitarian planner, K in constrained optimum $\gg$ competitive eqm.
   - Why? Overcome pecuniary externality: $K \uparrow \Rightarrow w \uparrow, R \downarrow$, redistribute from rich to poor (high labor share).

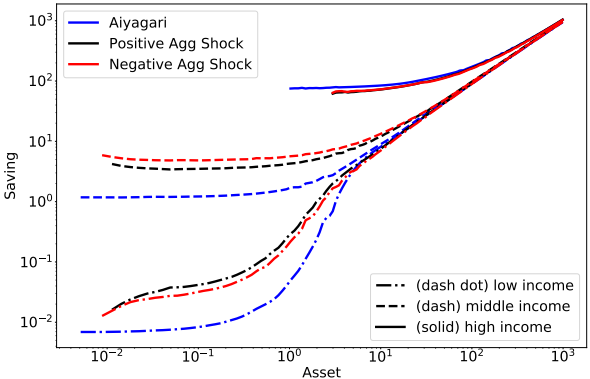
Constrained Efficiency for HA Models w or w/o Agg Shock

	No aggregate shock		Aggregate shock	
	Market	Constrained Opt.	Market	Constrained Opt.
Average assets	30.635	119.741	34.296	95.811
Wealth Gini	0.864	0.862	0.812	0.878
Consumption Gini	0.615	0.386	0.578	0.388

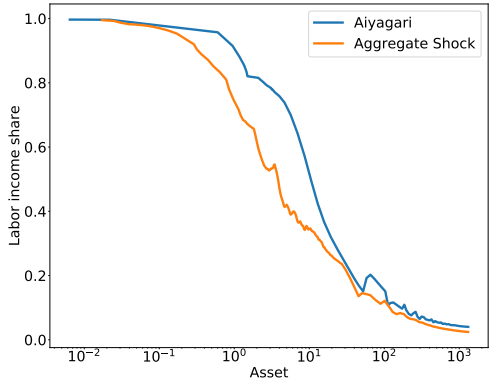
Findings:

1. Both models: with utilitarian planner, K in constrained optimum $\gg$ competitive eqm.
   - Why? Overcome pecuniary externality: $K \uparrow \Rightarrow w \uparrow, R \downarrow$, redistribute from rich to poor (high labor share).
2. Constrained optimal K in model w/ agg shock $<$ w/o agg shock.

Constrained Efficiency for HA Models w or w/o Agg Shock



(a) Policy function



(b) Labor share distribution

Agg shock $\Rightarrow$ precautionary saving $\uparrow$ by poor HHs $\Rightarrow$ labor share lower than model w/o agg shock. So planner raises K less in constrained efficient equilibrium.

The Agenda of "AI for Economics": Two Generations of Work

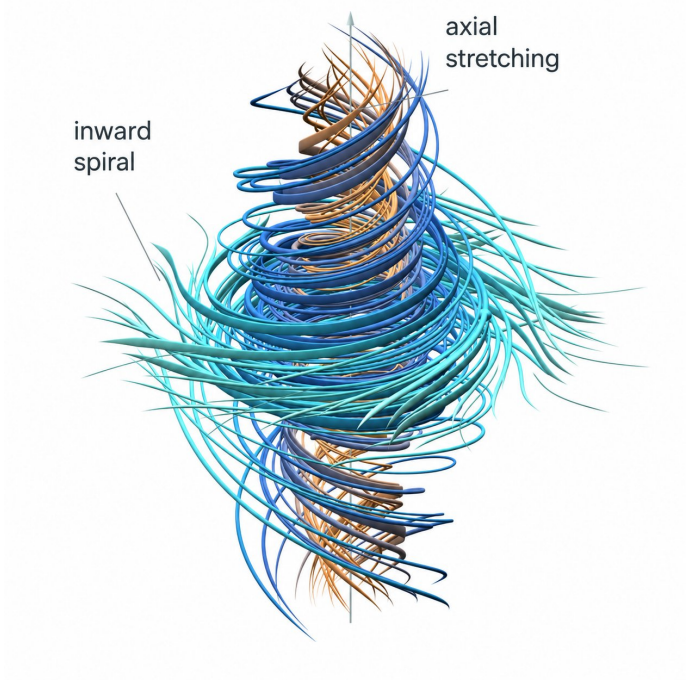
- First generation: "proof of concept" of using AI to solve economic models.
  - Duarte (2018), Azinovic, Gaegauf, Scheidegger (2019), Maliar, Maliar, Winant (2019), Fernández-Villaverde, Hurtado, Nuño (2019), Han, Yang, E (2021), Kahou, Fernández-Villaverde, Perla, Sood (2021), Valaitis and Villa (2021), Gopalakrishna (2022), Huang (2022), Gu, Lauriere, Merkel, Payne (2023), Zhong (2023), among others.
  - All papers solve similar classical models in this generation.
- Second generation: apply AI to long-standing difficult questions in various literature.
  - Climate economics: Folini, Friedl, Kubler, Scheidegger (2021), Friedl, Kubler, Scheidegger, Usui (2023), Barnett, Brock, Hansen, Hu, Huang (2023), Sun (2023).
  - Labor and financial markets: Payne, Rebei, and Yang (2026) $\Rightarrow$ Lecture 3.
  - Asset pricing: Zhang (2022), Azinovic-Zemlicka (2023), Gopalakrishna, Gu, Payne (2024).
  - HANK models: F-V, Marbet, Nuño, Rachedi (2022), Kase, Melosi, Rottner (2022).
  - Network models: Carvalho, Covarrubias, Nuño (2025).
  - Many other fields: trade, optimal policy, information, structural estimation, etc.

Summary

- Three elements of using deep learning to solve economic models:
  1. Parameterize the functions of interest as neural networks, can impose economic restrictions in parameterization
  2. Specify the objective
  3. Choose where to evaluate the objective (sampling)

- Brunnermeier et al (2021, RFS): “Neural network techniques will open up a new research avenue in macro-finance.”

- As is happening in other disciplines (Nobel Prize Physics 2024, Chemistry 2024, Navier-Stokes Solution 2026), AI may become the new generation of model solvers in economics.


axial
stretching
inward
spiral

DeepHAM Tutorial

DeepHAM Tutorial

Hands-on with the DeepHAM code on Google Colab (five notebooks, no installation):
github.com/yangycpku/Machine_Learning_Macro_Duke → Tutorials/Tutorial1

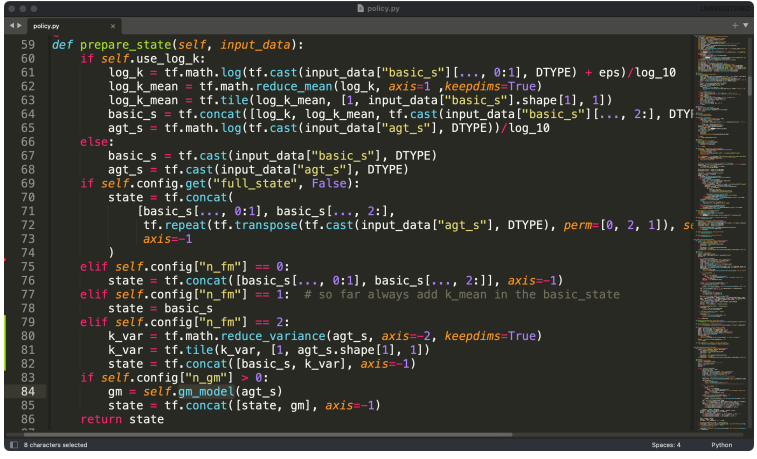
- 01: KS with the fixed moment (mean K); 02: one learned generalized moment instead
- 03: write the policy objective yourself (prices, budget constraint, `stop_gradient`, unrolled utility); 04: solutions
- 05: what the generalized moment learned: basis function $\mathcal{Q}$, value function, DeepHAM vs. KS dynamics
- `RUN_MODE`: `smoke` (minutes, in class), `teaching` (GPU, $\sim$15 min), `production`

Next slides, key elements of the code: general procedure; generalized moments; value update (supervised learning); policy update on the simulated path; fictitious play vs. planner

General procedure: data preparation, value function, and policy function

```python
# initial value training
init_ds = KSInitDataSet(mparam, config)
value_config = config["value_config"]
if config["init_with_bchmk"]:
    init_policy = init_ds.k_policy_bchmk
    policy_type = "pde"
else:
    init_policy = init_ds.c_policy_const_share
    policy_type = "nn_share"
train_vds, valid_vds = init_ds.get_valuedataset(
    init_policy, policy_type, update_init=False
)
vtrainers = []
for i in range(value_config["num_vnet"]):
    config["vnet_idx"] = str(i)
    vtrainers.append(ValueTrainer(config))
for vtr in vtrainers:
    vtr.train(
        train_vds, valid_vds,
        value_config["num_epoch"], value_config["batch_size"]
    )

# iterative policy and value training
policy_config = config["policy_config"]
ptrainer = KSPolicyTrainer(vtrainers, init_ds)
ptrainer.train(policy_config["num_step"], policy_config["batch_size"])
```

Generalized moments

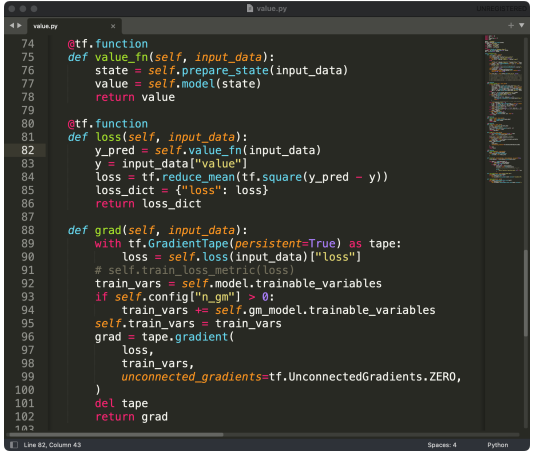


Generalized moments (Ct'd)

util.py

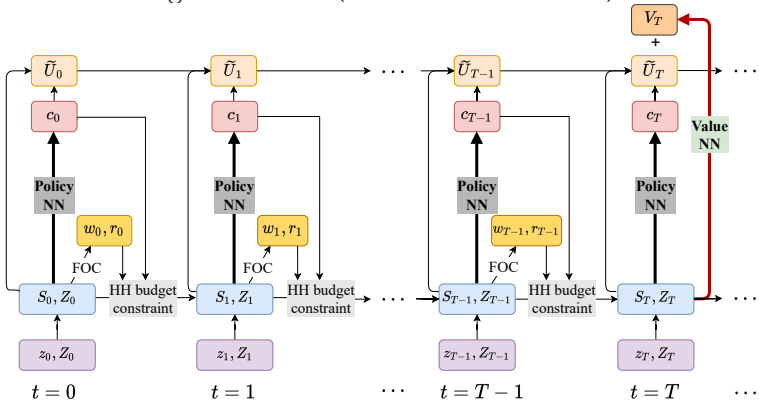
```
class GeneralizedMomModel(FeedforwardModel):
    def __init__(self, d_in, d_out, config, name="generalizedmomentmodel", **kwargs):
        super(GeneralizedMomModel, self).__init__(d_in, d_out, config, name=name, **kwargs)

    def basis_fn(self, inputs):
        x = self.dense_layers[0](inputs)
        for l in self.dense_layers[1:]:
            x = l(x)
        return x

    def call(self, inputs):
        x = self.basis_fn(inputs)
        gm = tf.reduce_mean(x, axis=-2, keepdims=True)
        gm = tf.tile(gm, [1, inputs.shape[-2], 1])
        return gm
```

Value Function Updating



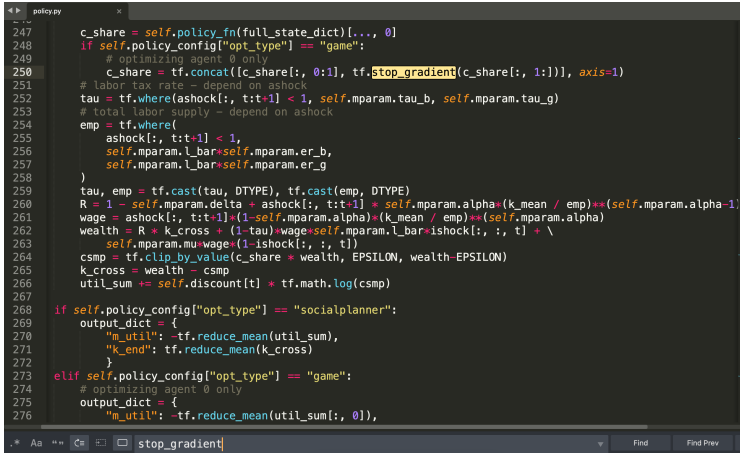
Recall: Computational Graph for Policy Function Optimization

$$\max_{\Theta^C} \mathbb{E}_{\mu(\mathcal{C}^{(k-1)}),\mathcal{E}} \left(\widetilde{U}_{i,T} + \beta^T V_{\text{NN}}(s_{i,T}; \Theta^V) \right)$$



Budget constraint $a_{i,t+1} = R_t a_{i,t} + w_t y_{i,t} - c_{i,t}$. $s_t = (a_{i,t}, y_{i,t}, Z_t, \mathbf{\Gamma}_t)$. Cumulative utility $\widetilde{U}_{i,t} = \sum_{\tau=0}^{t} \beta^\tau u(c_{i,\tau})$

Policy Function Updating: Cumulative Utility and Fictitious Play



Thank You!

Comments and questions are welcome!

My email: yucheng.yang@uzh.ch

Appendix: Backpropagation back

- Layers $z^{(p)} = W^{(p)} h^{(p-1)} + b^{(p)}$, $h^{(p)} = \sigma^{(p)}(z^{(p)})$; for sample i, $L_i \equiv L(x_i; \Theta)$ and $\delta^{(p)} \equiv \partial L_i / \partial z^{(p)}$
- Forward pass: evaluate once, store all $z^{(p)}$, $h^{(p)}$; then backward recursion

$$\delta^{(p-1)} = \left(W^{(p)\top} \delta^{(p)}\right) \odot \sigma^{(p-1)\prime}(z^{(p-1)}), \qquad \frac{\partial L_i}{\partial W^{(p)}} = \delta^{(p)} h^{(p-1)\top}, \qquad \frac{\partial L_i}{\partial b^{(p)}} = \delta^{(p)}$$

- One matrix product per layer: gradient costs a few times one evaluation
- Reverse-mode automatic differentiation (PyTorch, JAX, TensorFlow): same idea for any program; also gives $\partial \widehat{f} / \partial x$ for Euler equations
- Nonconvex: no global-optimum guarantee, seed-dependent $\Rightarrow$ try several seeds

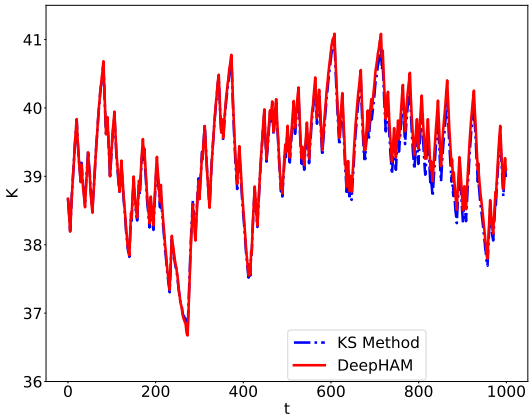
Appendix: Why Do Neural Networks Work? back

No complete theory yet; three ingredients:

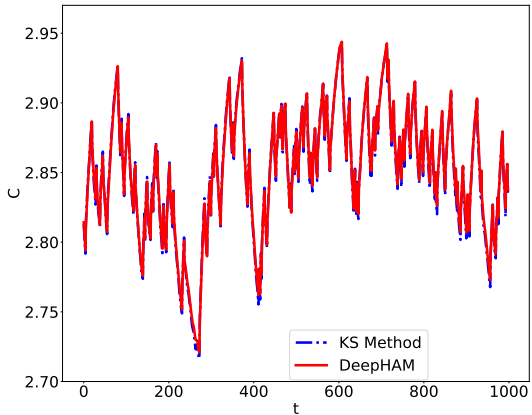
- Approximation: Barron (1993): H adaptive units $\Rightarrow$ error $O(1/H)$ regardless of d; fixed basis with H terms: at best $O((1/H)^{2/d})$
  - adaptive basis aligns with the directions along which f varies
  - universal approximation (Cybenko '89, Hornik et al. '89): existence only, silent on width
- Optimization: backprop makes exact gradients cheap; overparameterization eases descent; SGD picks smooth solutions among many fits (implicit regularization)
- Computation: matrix products + minibatch SGD parallelize on GPUs; free autodiff

Caveat: on generic smoothness classes, same curse of dimensionality as other methods

Solution Comparison



(a) aggregate capital (K_t)



(b) aggregate consumption (C_t)

back

Accuracy Measures: Bellman Equation Errors

For the KS problem, only using solved value function $V(\cdot)$, Bellman equation error is

$$\text{err}_{\text{B}} = V(a_i, y_i, Z, \mathbf{a}^{-i}, \mathbf{y}^{-i}) - \max_{c_i} \left\{ u(c_i) + \beta \sum_{y', Z', \mathbf{y}'^{-i}} V(a_i', y_i', Z', \widehat{\mathbf{a}'}^{-i}, \mathbf{y}'^{-i}) \times \Pr\left(Z', y'^{i}, \mathbf{y}'^{-i} | Z, y^{i}, \mathbf{y}^{-i}\right) \right\}$$

back